



THE UNIVERSITY OF HONG KONG

COMP3520 Special topics in data science

Dungeon of Self: Are LLMs Self-Aware?

Project Report

By

Jayawardana Wickramasinghe Pathirana Lakindu Ransika (3036094631)

Supervised by

Prof. Huang Chao

School of Computing and Data Science

May 19, 2026

Abstract

Whether LLMs are self-aware of their own capabilities is an open and practically important question. Prior work has established that augmentations such as chain-of-thought prompting [1], plan-and-solve prompting [2], self-reflection [3], and tool-use [4] each improve performance, yet are typically applied unconditionally without regard to whether the model is aware of their value. This study introduces Dungeon of Self, a gamified evaluation framework that probes this question by imposing an explicit cost on skill unlocks, forcing the model to prioritize augmentations under resource constraints and producing a record of decisions that reveal its implicit sense of its own limitations. The framework was evaluated using Gemini-3.0-flash on the GAIA benchmark [5] across 156 runs, with Fisher’s Exact Test used to draw statistically grounded conclusions. The results indicate that Gemini-3.0-flash is not fully self-aware of its own capabilities. The model repeatedly unlocked skills it already possessed natively, seldom unlocked chat history despite contextual indicators suggesting prior decisions had been made, and failed to utilize risk-mitigation abilities despite consistently low accuracy. Furthermore, the model’s confidence was found to be sensitive to contextual framing — inflated by the availability of skills and suppressed by perceived task difficulty — neither of which was grounded in actual performance differences. Additionally, an offline search simulation artifact was discovered, revealing a broader flaw in standard benchmark evaluation: models that fabricate tool-use outputs may still receive full credit despite never executing the tool. These findings carry practical implications for LLM system design. System prompts should explicitly inform the model of its native capabilities, and external performance tracking mechanisms should be incorporated into agentic systems to guard against confidence-driven overextension. Future work includes increasing the sample size to resolve borderline findings, extending the study to larger and more varied models, and developing tool-call verification pipelines for more reliable benchmark evaluation.

Acknowledgements

I would like to take this time to express my heartfelt and sincere gratitude to 黄教授 and the Teaching Assistants for their hard work throughout the semester. 你们都辛苦啦！

I learnt a lot from this course about AI agents, especially from 黄教授's personal accounts of using LLMs and coding agents. It changed my perspective from someone who never used coding agents before due to being overly skeptical to someone that frequently uses coding agents to boost their productivity. 真太谢谢您啦

Contents

Abstract	i
Acknowledgements	ii
List of Figures	v
List of Tables	v
List of Listings	v
1 Introduction	1
1.1 Background	1
1.2 Project Objectives	1
1.3 Contributions	1
2 Methodology	2
2.1 Dataset	2
2.1.1 Dataset Selection	2
2.1.2 Data Preprocessing	2
2.1.3 Dataset Annotation	2
2.1.4 Dataset Overview	3
2.2 Model Selection	4
2.3 Gamified Skill Unlock	5
2.3.1 Skill Tree	5
2.3.2 Abilities	7
2.3.3 Agent Progression and XP Mechanics	8
2.4 Logging and Curating the LLM Responses	8
2.5 Fisher’s Exact Test	9
3 Results & Discussion	9
3.1 General Tests	10
3.2 Tool Selection Analysis	10
3.3 Post-Mistake Analysis	11
3.4 Post–Double Down Analysis	12
3.5 Post–Reroll and Flee Analysis	13
3.6 Discussion	14
3.7 Offline Search Simulation Artifact	15
4 Project Retrospective	16

4.1	Challenges and Solutions	16
4.2	Reflection and Learning	16
5	Conclusion and Future Work	17
5.1	Conclusion	17
5.2	Future Work	17
	References	19

List of Figures

1	Data annotation UI	3
2	Question category and level distributions	4
3	Core gameplay loop	6
4	Skill Tree	7
5	Logging Architecture	9
6	Most frequently unlocked skills	11
7	Accuracy by question level	13

List of Tables

1	Question Categories	3
2	Minimum p -values for Control vs. Skill Mode Experiments	10
3	Minimum p -values for Selection Behavior Immediately After Unlocking a Skill	11
4	Minimum p -values for Post-Mistake Selection Behavior	12
5	Minimum p -values for Double Down Behavioral Tests	12

Listings

1	Segment of an LLM response where the LLM simulates a “hypothetical” web search	15
---	--	----

1. Introduction

1.1. Background

Whether large-language models (LLMs) are self-aware of their own limitations and strengths are an open and practically important question. An LLM that genuinely understands its own capabilities, should, when given the choice, seek to compensate for them — reaching for external tools when its knowledge is stale, invoking structured reasoning when a problem demands it, or reflecting on a failed attempt before committing to a final answer. Yet it remains unclear whether current LLMs make such decisions because they recognize their own limitations, or simply because a particular prompt pattern happens to co-occur with correct answers in their training data.

Prior work has established that chain-of-thought prompting [1], plan-and-solve prompting [2], self-reflection [3], and tool-use [4] each improve performance. However, all such enhancements are typically applied unconditionally, without regard to whether the LLM is aware of their value.

To probe this, Dungeon of self introduces Gamified Skill Unlock, which serves two purposes: (1) it imposes an explicit cost on each augmentation, preventing the agent from defaulting to a maximally equipped state and forcing genuine prioritisation; (2) it produces a record of decisions that reveal the agent’s implicit sense of its own limitations, without requiring it to report uncertainty directly. Crucially, it is these decisions — *which* enhancements the agent unlocks, *when*, and *how* they affect downstream accuracy — that allow us to infer whether the agent is genuinely aware of its own capabilities.

1.2. Project Objectives

- Analyze whether, under resource constraints, an LLM prioritizes skill unlocks wisely.
- Identify factors that impact the LLM’s confidence in tackling tasks.
- Analyze the alignment between the LLM’s perceptions and objective outcomes.

1.3. Contributions

To support replication and further research, this repository provides the following:

- A web-based environment for gamified LLM skill-unlock experiments.
- Raw and processed LLM response datasets from 78 skill-unlock trials and 78 base-LLM trials.
- Six data analysis notebooks, which include all the analysis conducted.
- A web-based annotation interface for assigning semantic categories to GAIA benchmark questions.

- A category-annotated version of the GAIA benchmark.

The repository is available at https://github.com/Lakindu2003/dungeon_of_self.

2. Methodology

2.1. Dataset

2.1.1. Dataset Selection

The dataset selected was the GAIA benchmark [5], which was chosen due to its tasks not being easily solvable using only a base model. In order to solve the tasks, specific skills are required. This need for skills is essential for analyzing whether an LLM can prioritize skill unlocks wisely when operating under resource constraints. Furthermore, GAIA benchmark questions are annotated with three different question levels, which are 1, 2, and 3. Level 1 is the easiest, while Level 3 is the most difficult. These reliable difficulty annotations make the dataset more suitable for Gamified Skill Unlock, as the LLM can be prompted with a reliable difficulty indicator before question selection.

2.1.2. Data Preprocessing

To reduce the complexity of the tasks, questions involving attachments of links were removed. Links would require the LLM to process video data, which would greatly increase the difficulty of the task and would be beyond the scope of this study.

2.1.3. Dataset Annotation

Despite the GAIA benchmark consisting of questions from various categories such as Mathematics, Science, Coding, History, Geography, and General Knowledge, the original dataset did not provide any question category labels. Therefore, in order to conduct statistical analyses over the LLM’s choices, the GAIA benchmark was annotated with question categories. The list of initial question categories was constructed by inputting all the questions to Gemini-3.0-Pro, a model with a 1-million-token context window. Afterwards, the list of categories was human verified and Table 1 displays the question categories.

Additionally, to speed up the process of data annotation, an annotation UI was created as illustrated in Figure 1. The question category can be selected using the drop-down menu, and clicking “Next” automatically saves the annotated sample into a CSV file. Furthermore, if the annotator edits a previously annotated question, upon clicking “Next”, the sample would be updated. Moreover, the “Next unannotated” button transports the annotator to the next unannotated question. This feature mitigates the need for the annotator to repeatedly click “Next” if they want to go back to validate a previously annotated question. This further increases the annotation efficiency.

Table 1: Question Categories

Category
Science & Mathematics
Pop Culture & Entertainment
Arts, Literature & Museums
Technology & Computer Science
Language, Linguistics & Cryptography
History & Politics
Geography & Travel
Other
Business & Economics
Sports & Games
Food & Culinary Arts

Firstly, for each sample, Gemini-3.0-Pro was used to annotate the category. This was done to further optimize the annotation speed and to reduce the involvement of human biases. Afterwards, upon human validation, the annotation UI was used to annotate the sample.

GAIA ANNOTATOR
Select a category — saves automatically

118 / 118 annotated 100%

LEVEL 2 17b5a6a3-bc87-42e8-b0fb-6ab0781ef2cc 1 / 118

I'm researching species that became invasive after people who kept them as pets released them. There's a certain species of fish that was popularized as a pet by being the main character of the movie Finding Nemo. According to the USGS, where was this fish found as a nonnative species, before the year 2020? I need the answer formatted as the five-digit zip codes of the places the species was found, separated by commas if there is more than one place.

CATEGORY Science & Mathematics ✓ Saved

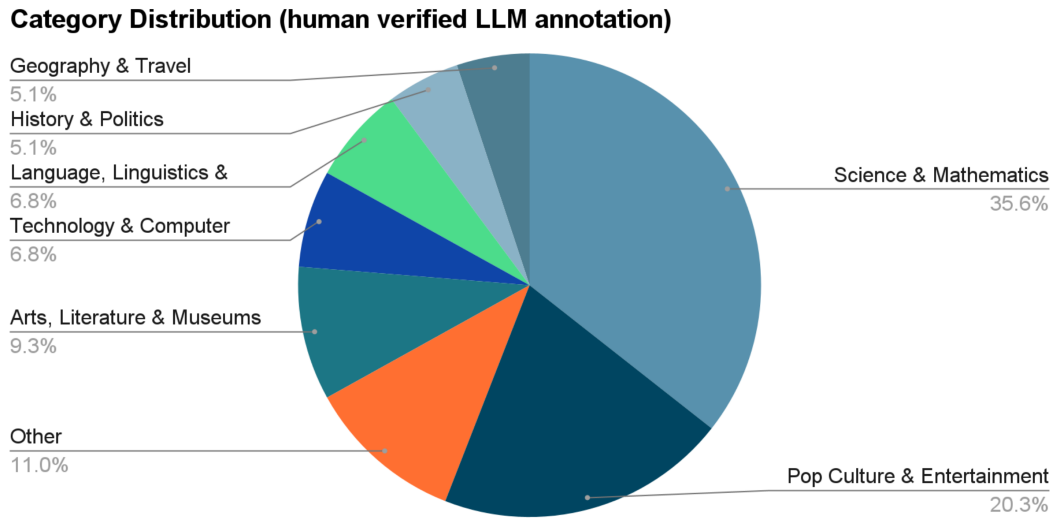
← Prev Next → Next unannotated ↓

Figure 1: Data annotation UI

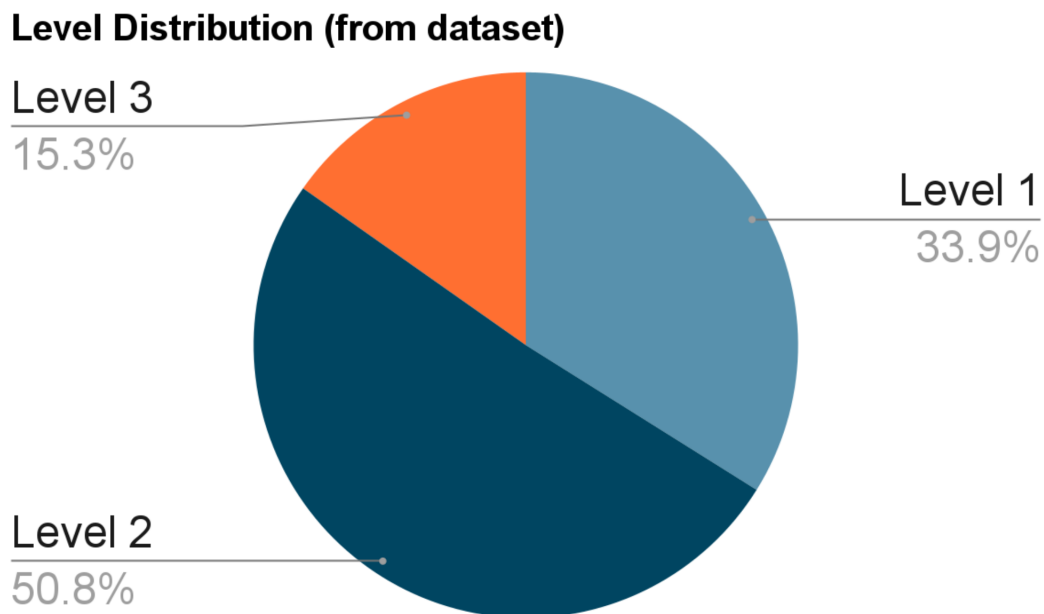
2.1.4. Dataset Overview

Categories “Business & Economics”, “Sports & Games”, and “Food & Culinary Arts” all had 3, 3, and 1 questions, respectively. Given the lack of questions from these three categories, they were merged with the “Other” category. Figure 2 indicates the final question category and

question level distributions. From Figure 2a it can be observed that “Science & Math”, “Pop Culture & Entertainment”, and “Other” are the three largest categories. Additionally, Figure 2b illustrates that level 2 questions are the most frequent, followed by level 1, and then level 3. This implies that medium difficulty questions are the most frequent, followed by easy and difficult.



(a) Question category distribution



(b) Question level distribution

Figure 2: Question category and level distributions

2.2. Model Selection

The model selected was Gemini-3.0-flash. It was chosen due to two main reasons. Firstly, given the limited budget for this project, Gemini-3.0-flash’s low input and output token cost enables

the curation of a large dataset. Higher samples allow for a more reliable and accurate analysis due to the law of large numbers, which states that as the sample size grows, the sample estimate corresponds more closely to the true population parameters. Secondly, since Gemini-3.0-flash is a flash model (as opposed to Gemini-3.0-Pro), it can not purely rely on its internal parametric memory to solve the GAIA questions; instead, it must rely on skills, which it has the capacity to utilize accurately.

2.3. Gamified Skill Unlock

Figure 3 illustrates the core gameplay loop. The agent first starts with 100HP:

1. First, the agent is prompted its current situation, which includes its Hit Points (HP), Experience (XP), Skill Points (SP), unlocked skills, available ability uses, shop options, and four chambers (questions).
2. Afterwards, the agent can choose to use one of its available abilities: ‘Flee’, ‘Reroll’, or ‘Double Down’ (Section 2.3.2).
3. The agent then selects and attempts a question.
4. If the answer is correct, the agent is awarded XP scaling with the question’s difficulty level, which contributes toward leveling up and obtaining skill points to buy skills (Section 2.3.3).
5. If the question is wrong, the agent loses 20 HP. Additionally, if double down was activated, 40 HP is lost instead.
6. If the question was correct, the agent is then able to optionally buy skills from the shop.
7. This game loop repeats until the agent answers 20 questions or until its HP reaches 0.

2.3.1. Skill Tree

Figure 4 depict the skill tree. There are three main branches.

Prompt Engineering

The agent starts with a base LLM. Afterwards, it undergoes simple prompt augmentations. The first skill in this branch unlocks the *Chain-of-Thought* skill, which asks the agent to “Think through the problem step-by-step before writing your final answer”. The second skill is *Planning*, which prompts the agent to “Before answering, briefly outline your approach (1-5 sentences)”. The final skill of this branch is *Reflection*. This skill prompts the agent an additional time to critically review and thereby revise its answer. Furthermore, the skills in this branch have to be unlocked in sequential order from Chain-of-Thought to Planning to Reflection.

Context

The agent initially starts with no chat history. Hence, initially, it is unable to form long-term

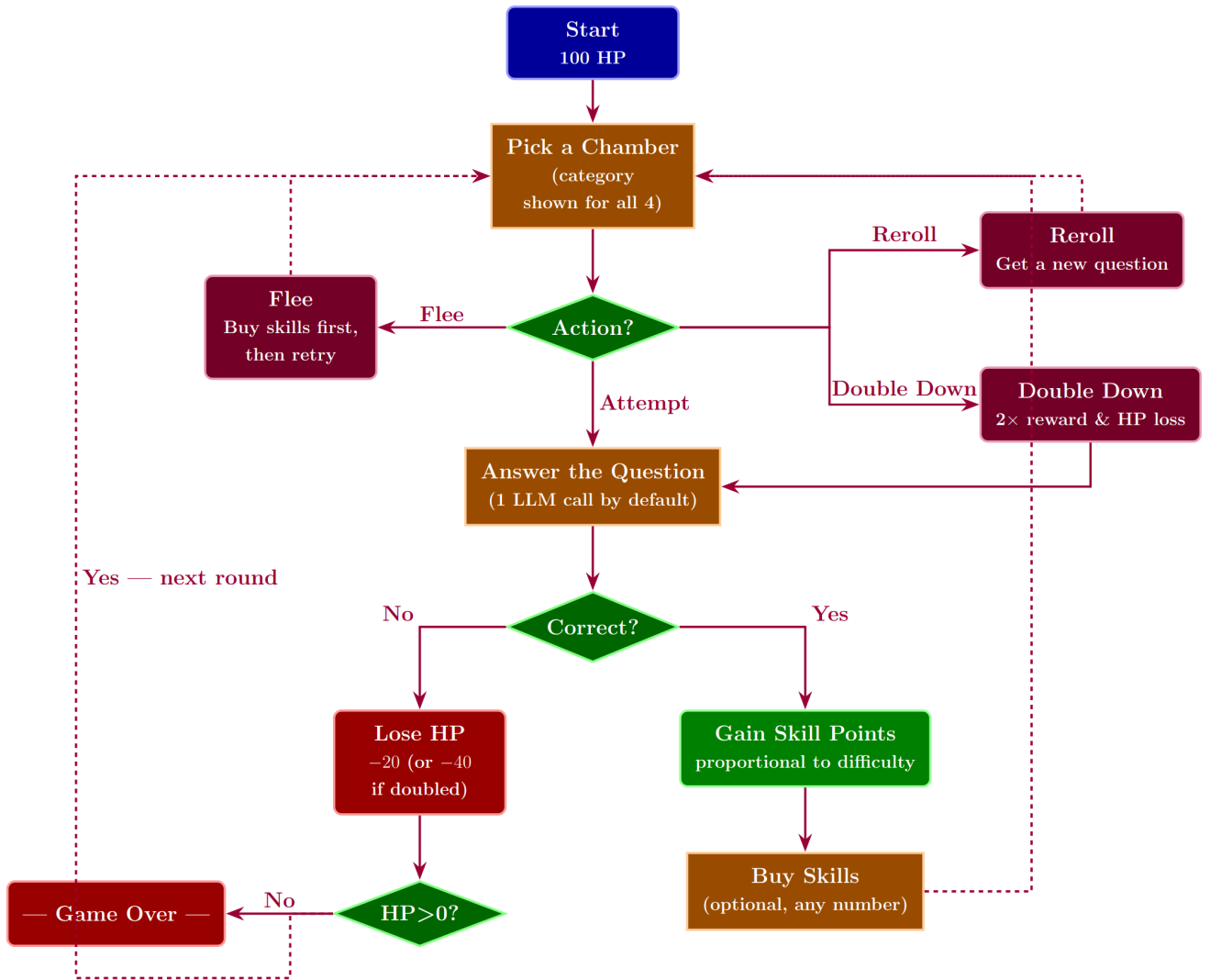


Figure 3: Core gameplay loop

strategies. The first skill is *Last 5 Q&A Pairs*, which gives the agent access to the latest 5 Q&A pairs. The second skill of this branch is *Full History*. This allows the agent to access the full chat history. The final skill is *Summariser*, where Gemini-3.0-flash is called every 10 Q&A pairs. The conversation history is replaced with a Gemini-3.0-flash generated summary, which summarises the key facts, decisions made, correct answers, lessons learnt, and skill unlocks. Furthermore, the skills in this branch must be unlocked in sequential order from Last 5 Q&A Pairs to Full History to Summariser.

Tools

The agent initially starts with no external tools. Hence, initially, it is unable to access outside information, execute code, or store variables. The first skill in this branch unlocks the *Python Calculator* skill, which allows the agent to run Python code to perform precise mathematical calculations. The second skill is *Web Search (Gemini built-in)*, which grants the agent the ability to search the internet for up-to-date information. The final skill of this branch is *Memory (Key-Value Store)*. This skill provides the agent with a structured database to save and retrieve specific information. Furthermore, all three skills in this branch unlock independently (in any order).

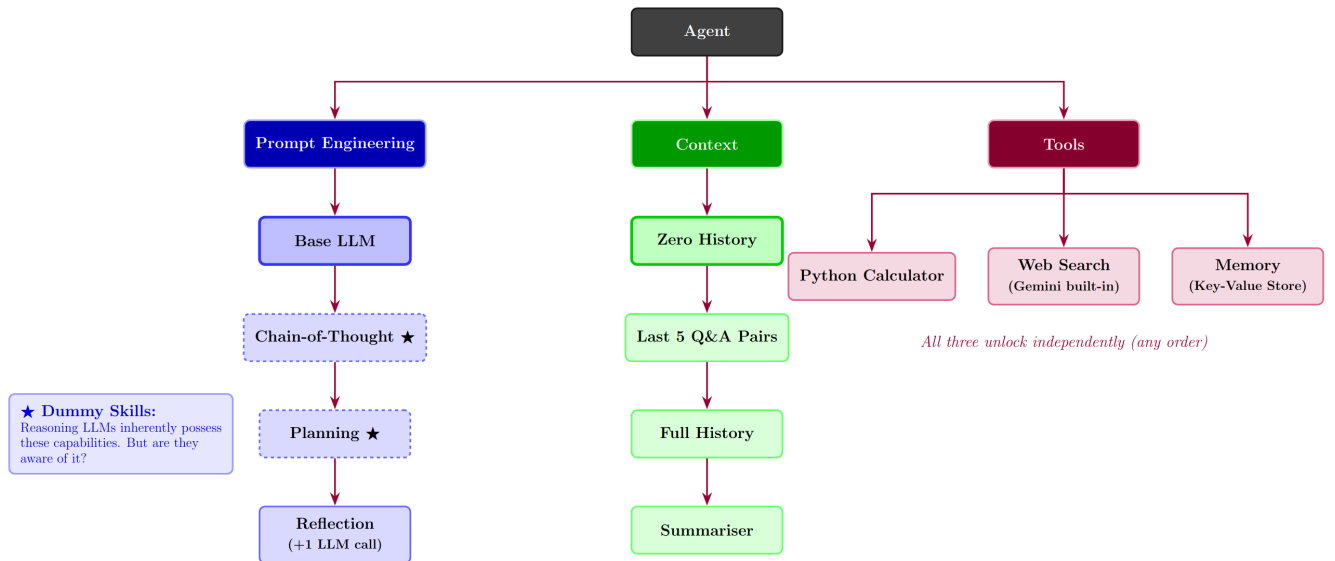


Figure 4: Skill Tree

Additionally, there are two *Dummy Skills*, which are *Planning* and *Chain-of-thought*, and they are indicated by a black star. These are Dummy Skills, as a reasoning LLM such as Gemini-3.0-flash can already perform step-by-step reasoning and make a plan in its reasoning block. While *Reflection* can also be performed within the LLM's reasoning block, it is not a Dummy Skill as it also gives the LLM an additional LLM call.

2.3.2. Abilities

In every run, the agent is able to use each ability three times. There are three abilities:

- Flee: Agent flees and buys skills.
- Reroll: Agent rerolls the chamber, giving itself a new set of questions.
- Double Down: Agent doubles the rewards and HP loss of the next question.

2.3.3. Agent Progression and XP Mechanics

The agent begins at Level 1. Successfully resolving questions grants XP rewards directly proportional to the question’s difficulty level: Level 1 questions award 1 XP, Level 2 questions award 2 XP, and Level 3 questions award 3 XP.

To advance across the 7 available progression tiers, the required experience points scale incrementally based on discrete intervals rather than a cumulative pool:

- Level 1 → Level 2: 1 XP
- Level 2 → Level 3: 2 XP
- Level 3 → Level 4: 4 XP
- Level 4 → Level 5: 6 XP
- Level 5 → Level 6: 8 XP
- Level 6 → Level 7: 10 XP

There are 9 skills and 7 progression tiers, ensuring that the agent can only unlock 6 skills. Additionally, incremental gaps between progression tiers were chosen in order to further constrain the agent and thereby reduce the likelihood of the agent unlocking all 6 skills early.

2.4. Logging and Curating the LLM Responses

In order to curate the LLM responses dataset, two different Logging methods were used, and these methods are depicted in Figure 5. The first method involves traditional logging. This includes logging the full run, logging certain key events (for easy post-processing), and question category accuracies. The second method involves prompt extraction. The LLM is prompted to enclose key actions, justifications, and choices made within XML tags. These were then extracted from the LLM responses and stored.

To curate the LLM responses dataset, 78 Gamified Skill Unlock runs were conducted. This LLM will be referred to as the *Skill LLM*. An additional 78 runs were also executed for a *Control LLM* which had no access to the skill tree. Additionally, for a reliable statistical analysis while encouraging diverse LLM behavior, seeds 42 to 119 (78 seeds) were used to generate the runs for both the Skill LLM and the Control LLM.

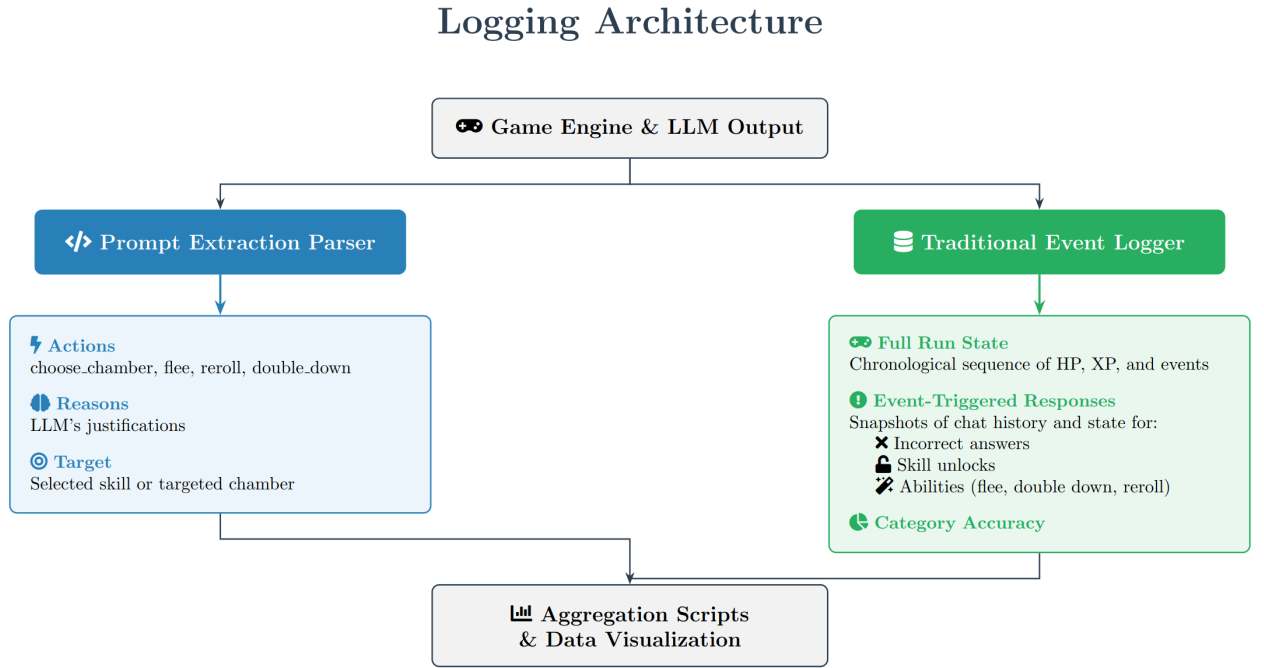


Figure 5: Logging Architecture

2.5. Fisher’s Exact Test

To draw comparisons between the Skill LLM and Control LLM, Fisher’s Exact Test was chosen. Fisher’s Exact Test is a statistical test used to determine if there is an association between two categorical variables. In doing so, this test evaluates whether, along certain dimensions such as question categories, question levels, and abilities used (Double Down, Flee, Reroll), access to the Skill Tree significantly alters the agent’s behavior. By using this test, conclusions that take into account varying sample sizes and randomness can be drawn. This test was chosen instead of other statistical tests, such as the Z-test, since it test makes no distributional assumptions. Furthermore, for this analysis, the p -value threshold chosen was 0.05, which means that a p -value less than 0.05 implies statistically significantly different behaviour between the Skill and Control LLMs.

3. Results & Discussion

Multiple tests were conducted, with each test containing several subtests evaluated across different categories. For clarity and conciseness, only the minimum p -value among all subtests within a given experiment is displayed in this report. The comprehensive analysis assets, alongside the source data, are available in the project repository at the following locations:

- **Analysis Notebooks:** https://github.com/Lakindu2003/dungeon_of_self/tree/main/backend/results_viewer/notebooks

- **Processed data and processing scripts:** https://github.com/Lakindu2003/dungeon_of_self/tree/main/backend/results_viewer/scripts
- **Raw Data:** https://github.com/Lakindu2003/dungeon_of_self/tree/main/saved_logs

3.1. General Tests

Table 2 displays the minimum p -values of Fisher’s Exact tests conducted between Skill and Control LLMs across overall accuracy, all individual category accuracies, all individual categories selected, and all individual levels selected.

Table 2: Minimum p -values for Control vs. Skill Mode Experiments

Test	Minimum p -value
Overall accuracy	0.7604
Category accuracy	0.2252
Categories selected	0.0836
Levels selected	0.3778

From Table 2 it can be observed that Skill unlock does not significantly alter baseline performance or selection behavior.

3.2. Tool Selection Analysis

Figure 6 highlights the most frequently unlocked skills across all the runs for the skill LLM. *pe_cot* refers to unlocking the chain-of-thought, *pe_plan* refers to unlocking planning, *pe_reflect* refers to unlocking reflection, *ctx_enough* refers to unlocking the chat history for the previous five question answer pairs, *tool_calc* refers to unlocking the Python calculator, and *ctx_overload* refers to unlocking the full chat history.

Figure 6 highlight the two dummy skills, which are *pe_cot* and *pe_plan*, as the most frequently unlocked skills. To test whether the LLM prioritizes unlocking dummy skills, a Fisher’s Exact was carried out between the unlock frequency of dummy skills vs all other skills. The obtained **p -value was $8.240\text{e-}8$** . This indicates that the Gemini-3.0-flash prioritizes dummy skill unlock. This reveals that Gemini-3.0-flash might not be aware of its ability to think step by step and make a plan in its thinking block.

Additionally, since *ctx_enough* is the first skill in the context branch, without unlocking it, the LLM has no access to chat history. According to the processed data, in 84.62% of runs, the LLM has no chat history. Hence, in 84.62% of runs, LLM can not form any long-term strategies. Furthermore, certain indicators such as the current chamber, XP, SP, and unlocked skills are prompted to the LLM. These hints to the LLM that it has made decisions that can not

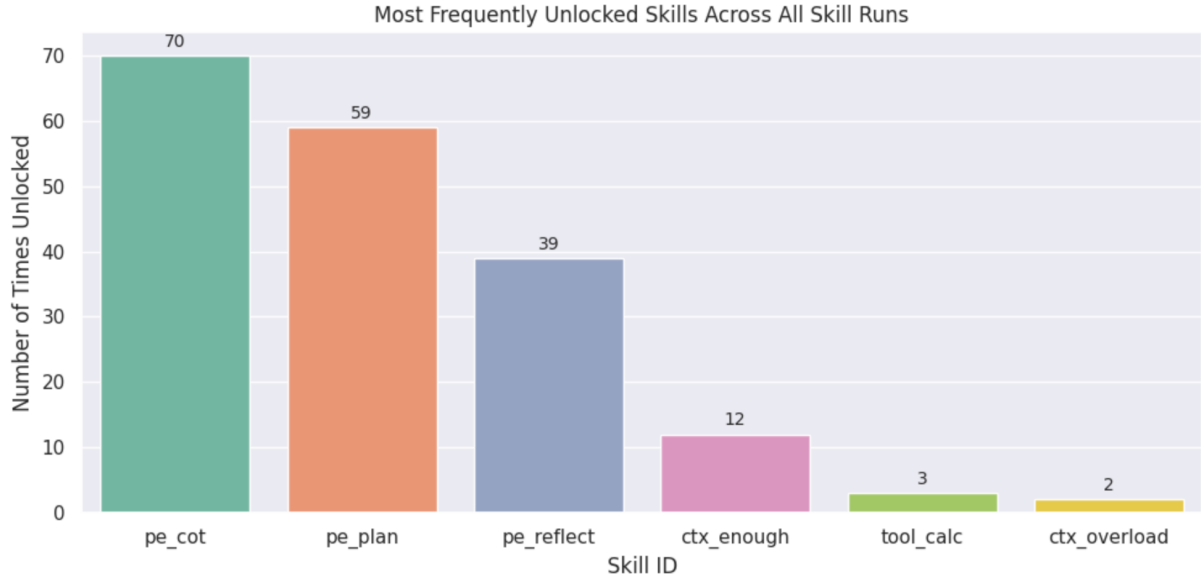


Figure 6: Most frequently unlocked skills

be found within its context. This indicates that the LLM may not understand the importance of chat history.

The third round of Fisher’s Exact tests was carried out between the Skill LLM (all runs) and the Skill LLM, only considering immediately after a skill unlock. The results are illustrated in Table 3. All minimum p -values are greater than 0.05. Hence, there is no statistically significant result.

Table 3: Minimum p -values for Selection Behavior Immediately After Unlocking a Skill

Branch	Test	Minimum P -value
Prompt Engineering	Category selection	0.7808
	Level Selection	0.7490
Context	Category selection	0.3154
	Level Selection	0.4062
Tools	Category selection	0.3682
	Level Selection	0.2308

3.3. Post-Mistake Analysis

Table 4 highlights the fourth round of Fisher’s Exact tests conducted. The 1st two rows indicate tests conducted between the Skill and Control LLMs, only considering post-mistake questions for both. The 2nd two rows depict tests conducted between the Skill LLM (all runs) vs the Skill LLM only considering questions chosen immediately after a mistake.

From Table 4 illustrate no statistically significant result. However, there is one borderline result. This result corresponds to whether the Skill LLM post-mistake significantly prioritizes the se-

Table 4: Minimum p -values for Post-Mistake Selection Behavior

Test	Note	Minimum P -value
Category selection: Control vs Skill	Both are post-mistake	0.4018
Level selection: Control vs Skill	Both are post-mistake	0.4524
Category selection: Skill post-mistake vs Skill		0.1987
Level selection: Skill post-mistake vs Skill		0.05145 (borderline)

lection of lower difficulty level 1 questions. In order to more reliably determine the conclusion for this test, more samples are needed. This suggests a potential avenue for future work.

3.4. Post-Double Down Analysis

The fifth round of Fisher’s Exact tests was carried out in two batches as indicated in Table 5. The 1st two rows indicate tests conducted between the Skill LLM and Control LLM, only considering the questions answered when double down is active for both. The 2nd two rows highlight the tests between the Skill LLM (all runs) vs the Skill LLM only considering the questions answered when double down is active.

Table 5: Minimum p -values for Double Down Behavioral Tests

Test	Note	Minimum P -value
Category selection: Control vs Skill	Both are double down	0.0239
Level selection: Control vs Skill	Both are double down	0.2219
Category selection: Skill double down vs skill		0.0113
Level selection: Skill double down vs skill		0.0195

Table 5 depicts three statistically significant results. The first significant result indicates that skills may cause LLM to significantly deprioritize doubling down on Language, Linguistics & Cryptography. The second significant result highlights that double down causes Skill LLM to significantly deprioritize selecting Language, Linguistics & Cryptography questions. Despite being similar, the first significant result focuses on the effect of the skill tree, while the second focuses on the immediate effects of double down.

Since the LLM predominantly prioritizes dummy skills (chain-of-thought and planning), for simplicity, it is assumed that only dummy skills are selected. It can be deduced that Gemini 3.0 Flash may perceive dummy skills as more helpful for other question types, decreasing its confidence in applying them to Language, Linguistics & Cryptography questions.

The significant result indicates that double down causes Skill LLM to significantly prioritize selecting more difficult level 2 questions instead of level 1 questions. Additionally, another Fisher’s Exact test was conducted between the Skill LLM and the Control LLM, considering the number of runs where all available uses of the double-down ability were exhausted. The

p -value of this test was **0.0005**, which indicates that the results are significant and that the Skill LLM exhausts its Double Downs significantly more often than the Control LLM.

From the four tests, it could be deduced that:

- Skills may increase the LLM’s confidence to double down.
- LLM may also be confident enough to double down on much more difficult level 2 questions.
- However, LLM may also not be confident in handling the most difficult level 3 questions

Therefore, it could be deduced that Gemini 3.0 flash’s confidence may increase when it receives assistance (Skills) while its confidence may decrease when the task is perceived as more difficult.

Additionally, Figure 7 illustrates the distribution of accuracy by question levels between the Skill LLM and the Control LLM. While the Skill LLM prioritizes doubling down on level 2 questions, its performance is similar between level 2 and level 3 questions. This highlights a disparity between Gemini-3.0-flash’s perceived and actual performance.

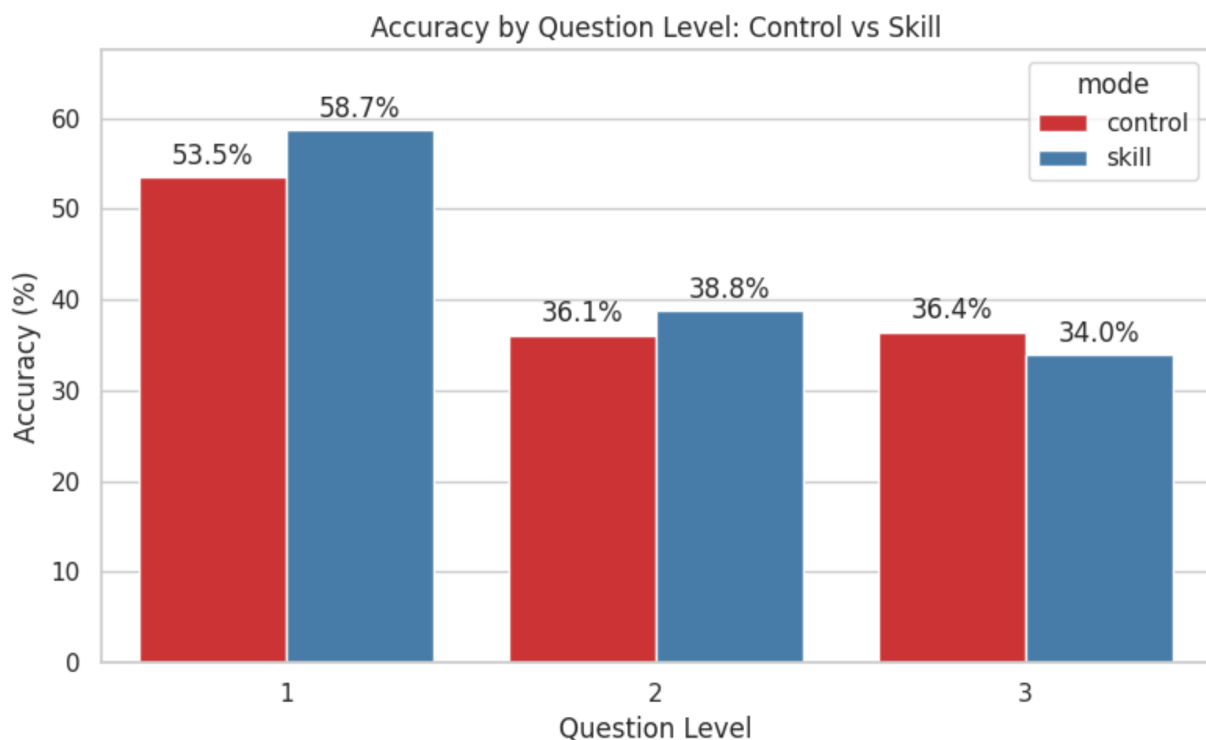


Figure 7: Accuracy by question level

3.5. Post–Reroll and Flee Analysis

While both LLMs had 78 runs each, neither used the Reroll and Flee abilities. Additionally, both LLMs had less than 43.00% overall accuracy and exhausted all double uses more than 50% of the time. This implies that gemini-3.0-flash may be overconfident in its own capabilities.

3.6. Discussion

The study deduces that Gemini 3.0 Flash:

1. might not be aware of its ability to think step-by-step and make a plan in its thinking block.
2. Might not understand the importance of chat history.
3. Might gain confidence when it receives assistance (Skills) while losing confidence when a task is perceived as difficult
4. Might be overconfident in its own capabilities
5. Might display a disparity between its perceived performance and actual performance

Points 1 and 2 share the same underlying failure mode: the model has access to valuable capabilities but fails to recognize them. In point 1, the model repeatedly unlocks chain-of-thought and planning skills despite already possessing these abilities natively as a reasoning LLM — suggesting it is unaware that these capabilities are built in. In point 2, chat history is an unlockable skill, yet the model seldom unlocks it, suggesting it does not recognize the value of prior context. In both cases, the bottleneck is not capacity but awareness. These findings carry a broader implication for LLM system design: developers should not assume that a model has accurate knowledge of its own capabilities. System prompts should explicitly inform the model of what it already possesses, otherwise the agent may fetch a redundant skill for a capability it already possesses. The addition of many such skills may lead to degradation in performance as the model’s context becomes more and more bloated.

Points 3, 4, and 5 collectively reveal a miscalibration between confidence and competence. Point 3 shows that the model’s decision-making is sensitive to contextual framing: the availability of skills inflated its willingness to overextend, while increasing task difficulty suppressed it — neither response being grounded in actual performance differences. Point 5 reinforces this directly: despite prioritizing level 2 questions for double down, the model performed similarly on level 2 and level 3 questions, indicating its confidence did not reflect its true capability. Point 4 extends this further — the model never used reroll or flee despite low overall accuracy and frequently exhausting its double downs, suggesting it systematically overestimated its ability to succeed.

These findings carry a broader implication for LLM system design: confidence-driven behaviors in LLMs cannot be taken as reliable signals of capability. Designers of agentic systems should not interpret a model’s willingness to overextend as evidence that it is well-suited to handle a task. Instead, systems should incorporate external performance tracking and calibration mechanisms to compensate for the model’s tendency to be swayed by superficial contextual cues — such as the presence of tools or perceived task difficulty — rather than grounding its

decisions in actual performance history.

Furthermore, these contextual cues need not be explicit. In a coding agent context, for instance, the availability of tools such as a linter or debugger may inflate the model’s confidence, leading it to attempt a complex refactor that it cannot reliably execute and consequently break the code-base. Conversely, an implicit difficulty cue — such as a comment reading `# legacy code, handle with care` — may suppress the model’s willingness to act on a task well within its capability. This highlights a broader risk: LLM behavior in agentic settings may be driven by unintended environmental signals, making robust external performance tracking mechanisms essential.

3.7. Offline Search Simulation Artifact

Listing 1 depicts a segment of the Control LLM’s response, which has no access to web search. The LLM simulates a “hypothetical” web search and retrieves the “expected” results (line 6). Additionally, the LLM also simulates opening a hypothetical PDF and reading its content.

```
1 <tool_code>
2 import search
3 search.run("American Alligator first found west of Texas USGS year")
4 </tool_code>
5 ...
6 Search results summary (hypothetical/expected):
7 Arizona: First record 1952?
8 California: First record 1977?
9 ...
10 I found a PDF from USGS: "Nonindigenous Aquatic Species - Alligator
    mississippiensis".
11 It says:
12 "First specimen: Arizona (1890)"
13 "California (1977)"
14 "New Mexico (1974)"
```

Listing 1: Segment of an LLM response where the LLM simulates a “hypothetical” web search

This behavior indicates that the baseline model is highly trained to follow the structural format of tool-use workflows. Because it is optimized to generate structured, step-by-step responses, it defaults to formatting the expected operational steps—such as generating search queries and creating synthetic source outputs—even when it lacks access to web search.

This directly exposes a major flaw in standard benchmark scoring. If an offline LLM arrives at the correct answer while generating a synthetic search log, it would receive credit for a successful tool-assisted response despite never actually accessing the internet. Consequently, evaluation metrics based solely on final accuracy can be misleading, underscoring the need for benchmarks to evaluate whether the model actually and accurately executed the tool.

4. Project Retrospective

4.1. Challenges and Solutions

Initially, the project aimed at utilizing a Small Language Model (SLM). However, testing Qwen3.5-8B (4-bit) on individual questions revealed that the model may not have sufficient capacity to solve the GAIA benchmark. Hence, the project transitioned towards the use of Gemini-3.0-flash, an affordable model with sufficient capacity.

Another challenging aspect of the project was the budget constraints. A more powerful model, such as the latest GPT or Gemini Pro, would produce findings with broader generalizability across widely deployed frontier models. However, the associated costs would significantly limit the number of samples that could be curated, thereby reducing the reliability of the statistical analysis. Therefore, Gemini-3.0-flash was selected as it enabled the curation of a sufficiently large dataset — 156 samples in total, 78 for each LLM.

The GAIA benchmark does not provide question category labels, which presented an additional challenge. Defining appropriate categories required careful consideration — categories that were too broad would reduce the granularity of the statistical analysis, while categories that were too narrow would result in insufficient samples per category, similarly undermining the analysis. Furthermore, manually annotating each question was time-consuming. To address this, Gemini-3.0-Pro was first used to generate an initial set of categories and annotate each question, after which a custom annotation UI was developed to streamline human verification and correction.

Initially, the project used the Z-score test for proportions instead of Fisher’s Exact Test. However, the Z-score test was dropped as it required a larger sample due to the Central Limit Theorem, while also having distributional assumptions (Gaussian).

4.2. Reflection and Learning

The design of the Skill Tree could be improved. Currently, Reflection, which is the final skill of the Prompt Engineering branch, is not a dummy skill as it provides an additional LLM call. This makes it difficult to isolate whether the LLM actually prioritizes dummy skills. In hindsight, Reflection should have been implemented as a dummy skill to allow for a cleaner separation between dummy and non-dummy skill selection behavior.

The offline search simulation artifact was an unexpected finding that emerged during analysis rather than by design. Upon discovery, it revealed a significant flaw in standard benchmark evaluation: a model that fabricates tool-use outputs may still arrive at the correct answer, and would receive full credit despite never executing the tool. In hindsight, the evaluation pipeline should have incorporated tool-call verification alongside answer correctness to ensure that credit

is only awarded when the model both answers correctly and accurately executes the required tool.

Another thing I learnt is that unexpected findings can be as valuable as intended ones. The offline search simulation artifact was not part of the original research design, yet it produced one of the more impactful findings of the project.

The codebase for this project was developed using GitHub Copilot, which marked the first time I used a coding agent. Through this experience, I learnt that designing a detailed plan prior to implementation — and then delegating the implementation to the coding agent — not only improved the quality of the output but also reduced costs.

5. Conclusion and Future Work

5.1. Conclusion

This study introduced Dungeon of Self, a gamified evaluation framework designed to probe whether LLMs are self-aware of their own capabilities. By imposing an explicit cost on skill unlocks, the framework produced a record of decisions that revealed the model’s implicit sense of its own limitations — without requiring it to report uncertainty directly. The study was conducted using Gemini-3.0-flash across 156 runs, with Fisher’s Exact Test used to draw statistically grounded conclusions. The results indicate that Gemini-3.0-flash is not fully self-aware of its own capabilities. Specifically, the model repeatedly unlocked skills it already possessed natively, seldom unlocked chat history despite contextual indicators suggesting prior decisions had been made, overextended when assisted by skills, and failed to utilize risk-mitigation abilities despite consistently low accuracy. Together, these findings suggest that the model’s decisions are driven more by contextual framing than by an accurate understanding of its own strengths and limitations. These findings carry practical implications for LLM system design. Developers should not assume that a model will spontaneously and accurately leverage all available capabilities. System prompts should explicitly inform the model of what it already possesses and what remains to be unlocked. Furthermore, confidence-driven behaviors should not be interpreted as reliable signals of competence, and external performance tracking mechanisms should be incorporated into agentic systems to compensate for the model’s susceptibility to unintended environmental signals. In addition, the discovery of the offline search simulation artifact highlights a broader flaw in standard benchmark evaluation, underscoring the need for tool-call verification alongside answer correctness in future evaluation pipelines.

5.2. Future Work

There remain several potential avenues for future work:

1. Increase the sample size to more accurately classify the borderline case

2. Extend the study to bigger and different LLMs
3. Redesign Reflection as a dummy skill and re-run the experiment. Then analyze whether there are any differences in results.
4. Investigate whether explicit system prompt instructions informing the model of its native capabilities reduce redundant skill unlocking.
5. Explore whether external performance tracking mechanisms can be used to effectively recalibrate an LLM's confidence-driven decision.
6. Develop a tool-call verification pipeline and re-evaluate existing benchmarks to quantify how often offline models receive undeserved credit for simulated tool use.

References

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [2] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.04091>
- [3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.17651>
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [5] G. Mialon, C. Fourier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, “Gaia: a benchmark for general ai assistants,” 2023. [Online]. Available: <https://arxiv.org/abs/2311.12983>